December stretch goals

Alan Guo | Aerospace 1C | November 6, 2026

Main goal: build a working web app

Our main stretch goal is to turn Compliance Copilot into a fully functional web app. Someone should be able to open the website, submit software requirements, and review the results without setting up Python or running a notebook.

We already have the RAG pipeline and tool-calling agent. The next step is to connect them to a clear interface, a backend that runs the audits, and a database that saves the results. That gives us something people can actually try during the final presentation.

What users should be able to do

- Paste a requirement or upload a CSV containing several requirements.
- Choose the baseline or agent, then start an audit and see its progress.
- Read each result: Meets, Partial, or Gap, with a short explanation.
- Open the cited NASA clause to understand what the result is based on.
- Save an audit and reopen it later. If multiple people use the app, each person should only see their own saved runs.
- Download the results as JSON or a readable Markdown report.

If an audit fails, the page should explain that it failed and let the user try again. It should never show a failed audit as a successful compliance result.

How we would build it

Start with one complete flow: enter a requirement, run the existing audit code, and display the result on the page. Once that works, add CSV uploads, saved history, and report downloads.

The main pieces are straightforward:

- The frontend is the part users see: the input form, progress indicator, results, and history.
- The backend receives requests from the page and runs our existing Python pipeline or agent.
- The database stores each run, its requirements, and its results so they can be reopened.
- The deployed app connects these pieces through one working website. Model credentials stay on the server.

We should reuse the current code wherever possible. The interface should make the existing system easier to use while keeping its explanations, citations, and error handling visible.

How we will know it works

For the first version, a teammate should be able to open the deployed website, submit a sample requirement, wait for a real audit, inspect the citation, refresh the page, and reopen the saved result. They should also be able to upload a small CSV and download the completed report.

We should test an invalid upload and a failed audit too. A working app needs to handle those cases clearly, not just display a successful example. The existing benchmark can help us check that connecting the website has not changed the audit results.

Other stretch goals

LangGraph: We could move the agent workflow into LangGraph to make its steps easier to follow and extend. This comes after the web app. Adding more agents would only be useful if our error analysis shows a specific problem that a separate reviewer could help with.

Support another standard: Later, users could choose a standard other than NASA NPR 7150.2D. This would require changes to how we store clauses, retrieve them, and display citations. We would also need examples checked against the new standard before claiming that it works.

Suggested order

Finish the current evaluation and error analysis alongside the web app work. Use those findings to decide which model improvements are worth pursuing next. The priority is a complete, usable interface; LangGraph and additional standards are follow-up options.

Nam and Shahreen are working on the baseline comparison, and Garima and Meryum are working on error analysis. We can use their results in the app and in the final presentation. We still need to agree on who will handle the frontend, backend, and deployment.

References

- Team planning document
- Existing project code
- LangGraph overview